

# Generative AI

**Nooreldin Lasheen**

e12302427@student.tuwien.ac.at

## Code and Repository

The source code for our final approach can be found here:

[Skillsync Github Repository](#)

## Abstract

Forming balanced teams for technical projects is challenging due to heterogeneous candidate backgrounds, noisy skill data, and implicit role requirements. We propose a team recommendation system that leverages Large Language Models (LLMs) to extract structured representations of candidate profiles and project needs from unstructured data. By embedding skills, experience, and collaboration traits into a shared semantic space, the system enables robust comparison despite schema inconsistencies and ambiguity. Team matching combines cosine similarity for candidate–project alignment with diversity measures, assembled via a multi-objective heuristic optimization approach. Crucially, the LLM provides semantic normalization and interpretive reasoning without overriding algorithmic decisions, ensuring explainability. Overall, the system reduces manual effort while facilitating transparent, human-centered decision-making in team formation rather than fully automating the process.

## 1. Data

We use data from the **Stack Overflow Annual Developer Survey 2024** (65,000 responses). The original 120 variables were reduced to 28 features aligned with the system’s extraction schema. A subset of 200 candidate profiles was selected to balance realism and computational efficiency.

Candidate data is stored at `data/skillsync_dataset_v1_draft_with_candidate_id.csv`. Unstructured project descriptions used for requirement extraction are provided at `data/sample/sample_projects.txt`.

**Source:** [Stack Overflow Annual Developer Survey](#)

## 2. Description

### 2.1. User Journey

This is explained in the video.

### 2.2. Technical approach

The system operates as a modular pipeline that guides users from raw data inputs to explainable team recommendations. Users provide a free-text project description and upload candidate data derived from the Stack Overflow survey. We utilize Large Language Models (specifically `gpt-4o-mini`) to extract structured representations into strict JSON schemas, ensuring fields like skills, experience, and collaboration traits are standardized.

To enable robust comparison, the system encodes these profiles and project requirements into a shared semantic vector space using the `all-MiniLM-L6-v2` sentence-transformer. This stages rely on carefully designed prompts and fixed JSON schemas, ensuring that all outputs conform to a standardized structure. This prompt-constrained extraction strategy reduces ambiguity and limits hallucinations by preventing the model from generating unsupported information.

Team formation employs a deterministic optimization strategy using greedy selection for single team or round-robin allocation for multiple teams to balance technical fit with team cohesion from a pool of candidates. Candidate selection is driven by a multi-objective scoring function:

$$\text{score} = \alpha \cdot \text{fit} + \gamma \cdot \text{coverageGain} + \beta \cdot \text{diversity} + \rho \cdot \text{roleGain} - \delta \cdot \text{overlapRedundancy}$$

Here, *fit* measures cosine similarity to the project, while *diversity* and *coverage* metrics ensure heterogeneous, well-rounded teams. Following construction, an LLM-based explainability layer generates natural-language reports justifying the recommendation without overriding the algorithmic decision.

### 2.3. Technical challenges

Key technical challenges included handling sparse behavioral signals in survey data—such as learning orientation and knowledge-sharing behavior—which resulted in a high proportion of missing evidence during extraction. Additional challenges arose from resolving entity inconsistencies (e.g., AWS vs. Amazon Web Services, GCP vs. Google Cloud Platform, as well as abbreviations and common variants). Furthermore, due to limited pre-processing, metadata fields (e.g., role metadata) were not sufficiently emphasized.

Another notable difficulty involved extracting programming language mentions, particularly those expressed as single-word tokens (e.g., C and R) or variants with inconsistent spacing, which were prone to being missed or misclassified.

These challenges were mitigated by enforcing verbatim evidence extraction and by designing prompts that explicitly anticipated downstream alignment constraints. To further address these limitations, consistency rules for alternative technology naming—including programming language variants were explicitly incorporated into the prompting logic.

### 3. Evaluation

The evaluation assesses whether the system produces grounded, reliable, and machine-interpretable representations of candidate profiles and project requirements using task-appropriate quantitative metrics.

#### 3.1. Candidate Profile Extraction

Candidate profile extraction is evaluated by comparing LLM-generated JSON profiles against the original input text. The evaluation focuses on schema validity, categorical label correctness, grounding of extracted skills and tools, and presence of short explanatory evidence spans.

The results show perfect schema validity, no invalid labels, and verbatim grounding of all extracted skills and tools, indicating strong resistance to hallucination. Lower evidence rates for knowledge sharing reflect limited explicit signals and conservative fallback to `Unknown`.

#### 3.2. Project Requirement Extraction

Project requirement extraction is evaluated on real-world project descriptions collected from online sources. Extracted skills, tools, and roles are manually curated to form ground truth and evaluated using precision, recall, F1-score, and accuracy.

High precision indicates strong support by project descriptions, while recall decreases for abstract or underspecified inputs.

#### 3.3. Latency

Latency results show near-linear scaling for candidate extraction, constant cost for project extraction, and input-dependent explanation latency.

#### 3.4. Discussion

The evaluation shows that the system reliably produces grounded and machine-interpretable outputs. Candidate profile extraction achieves perfect schema validity and strong resistance to hallucination, while project requirement extraction attains high precision and recall when descriptions are sufficiently explicit. Latency results indicate that the pipeline scales efficiently and supports interactive use.

A key limitation lies in the team matching stage. The current scoring function uses fixed weights and linear trade-offs, which may not fully capture project-specific priorities or complex interactions between technical fit and collaboration traits. Future work includes adaptive or learned weighting strategies and downstream validation of team quality. Despite this, the current approach provides a transparent and explainable baseline for team construction.

| <b>Metric</b>                            | <b>Value</b> |
|------------------------------------------|--------------|
| Schema validity rate                     | 1.00         |
| Invalid categorical labels               | 0            |
| Avg. skills verbatim-in-text rate        | 1.00         |
| Avg. tools verbatim-in-text rate         | 1.00         |
| Skill source-field failures (total)      | 38           |
| Tool source-field failures (total)       | 15           |
| Evidence present (personality role)      | 1.00         |
| Evidence present (communication style)   | 1.00         |
| Evidence present (conflict style)        | 1.00         |
| Evidence present (leadership preference) | 0.93         |
| Evidence present (deadline discipline)   | 1.00         |
| Evidence present (learning orientation)  | 1.00         |
| Evidence present (knowledge sharing)     | 0.61         |

Table 1: Candidate profile extraction results over 200 profiles.

| <b>Project</b>                   | <b>Precision</b> | <b>Recall</b> | <b>F1-score</b> | <b>Accuracy</b> |
|----------------------------------|------------------|---------------|-----------------|-----------------|
| Customer Loyalty Platform        | 0.750            | 0.500         | 0.600           | 0.429           |
| Smart Recommendation Marketplace | 1.000            | 0.833         | 0.909           | 0.833           |
| Virtual Events Platform          | 0.857            | 1.000         | 0.923           | 0.857           |
| Employee Performance Dashboard   | 1.000            | 0.667         | 0.800           | 0.667           |
| Recommendation-Based News App    | 1.000            | 1.000         | 1.000           | 1.000           |

Table 2: Overall extraction performance per project

| <b>Stage</b>               | <b>Latency (s)</b> | <b>Notes</b>                                                                                                              |
|----------------------------|--------------------|---------------------------------------------------------------------------------------------------------------------------|
| Candidate extraction (avg) | 10.4–15.9          | Measured per candidate profile; latency increases with the number of candidates processed.                                |
| Project extraction         | 0.02–4.9           | Independent of the number of candidates; exhibits short and approximately constant latency across runs.                   |
| Explanation generation     | 17.2–46.7          | Measured per candidate; latency increases with both the number of candidates and teams due to additional reasoning steps. |

Table 3: Pipeline latency (s)

## 4. Reflection

### 4.1. Human Agency

The system increases human agency by acting as a transparent decision-support tool. Explainable recommendations allow users to retain full control while using the system to navigate complex candidate and project data.

### 4.2. Future Development

Yes, the system could be extended by incorporating dynamic feedback from project outcomes and operational constraints such as availability and time zones, enabling adaptive matching and more realistic team formation.

## References

- [1] Stack Overflow (2024). *2024 Annual Developer Survey*. <https://survey.stackoverflow.co/>